



07-DARS · 14 TADAN

Git va GitHub

Versiyalarni boshqarish: commit, branch, merge, konfliktlarni hal qilish, remote repozitoriya, Pull Request, jamoaviy ish oqimi va GitHub Pages.

Boshlang'ich–O'rta · ~90 daqiqa

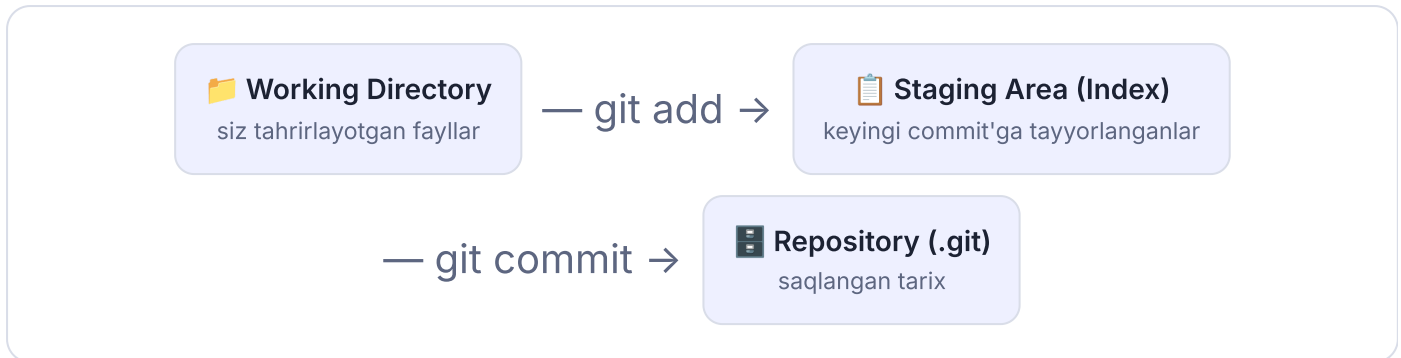
Tasavvur qiling: loyihangiz ishlab turibdi, siz yangi funksiya qo'shdingiz va hammasi buzildi. Qaysi o'zgarish sababchi? Kecha ishlagan versiyani qanday qaytarish mumkin? Uch kishi bitta faylni bir vaqtda qanday tahrirlaydi? **Git** — bu savollarning barchasiga javob beradigan **versiyalarni boshqarish tizimi (VCS)**. Bugun Git'ni bilmaydigan dasturchini ishga olishmaydi. Siz hozir o'qiyotgan qo'llanma ham Git orqali GitHub'ga joylangan.

Git va GitHub — farqi

- **Git** — kompyuteringizda ishlaydigan dastur. Loyiha tarixini saqlaydi. Internetsiz ham ishlaydi. 2005-yilda Linus Torvalds (Linux yaratuvchisi) yozgan.
- **GitHub** — Git repozitoriylarini internetda saqlash va jamoa bo'lib ishlash platformasi (Pull Request, Issues, Actions, Pages). Muqobillari: GitLab, Bitbucket.

Git'ning asosiy tushunchalari

Uch hudud



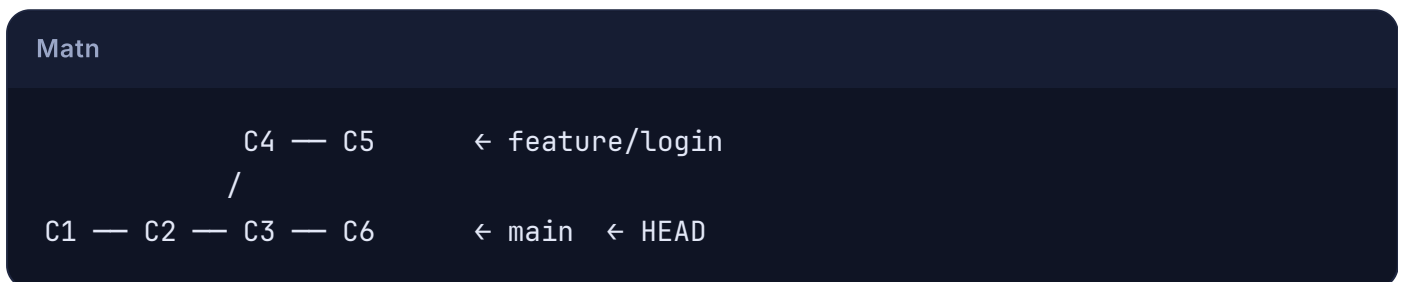
- **Working directory** — oddiy papka va fayllaringiz.
- **Staging area** — "savat": qaysi o'zgarishlar keyingi suratga (commit'ga) kirishini tanlaysiz. Bu bir nechta faylni o'zgartirib, ularni mantiqiy jihatdan alohida commit'larga bo'lish imkonini beradi.
- **Repository** — `.git` yashirin papkasi: barcha commit'lar, branch'lar va tarix shu yerda.

Commit

Commit — loyihaning ma'lum paytdagi to'liq **surati (snapshot)**. Har bir commit'da: noyob identifikator (SHA-1 xesh, masalan `a3f9c2e`), muallif, sana, xabar va ota commit'ga havola bor. Commit'lar zanjiri — loyiha tarixi.

Branch (tarmoq)

Branch — commit'ga ko'rsatkich (yorliq). Yangi branch yaratish — tarixdan "nusxa" olib, asosiy kodga tegmasdan tajriba qilish demak. `HEAD` — hozir qaysi branch'da turganingizni bildiruvchi maxsus ko'rsatkich.



O'rnatish va sozlash

Terminal

```
# Versiyani tekshirish
git --version

# Bir martalik sozlash (har bir commit'da ism va email yoziladi)
git config --global user.name "Ismingiz Familiyangiz"
git config --global user.email "siz@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"      # VS Code muharrir sifatida

git config --list    # sozlamalarni ko'rish
```

Asosiy buyruqlar

Terminal

```
# Yangi repozitoriya
git init

# Holat: qaysi fayllar o'zgargan, qaysilari staging'da
git status
git status -s          # qisqa ko'rinish

# O'zgarishlarni ko'rish
git diff               # working directory vs staging
git diff --staged      # staging vs oxirgi commit

# Staging'ga qo'shish
git add index.html     # bitta fayl
git add src/           # papka
git add .              # barcha o'zgarishlar
git add -p             # o'zgarishlarni bo'lakma-bo'lak tanlash (juda foydali!)

# Commit
git commit -m "Bosh sahifaga navigatsiya qo'shildi"
git commit -am "Xabar" # kuzatilayotgan fayllarni add + commit (yangi fayllar kirmaydi)

# Tarix
git log
git log --oneline --graph --all  # chiroyli daraxt ko'rinishi
git log -p index.html           # fayl o'zgarishlari tarixi
git show a3f9c2e                # bitta commit tafsilotlari
git blame app.js                # har bir qatorni kim, qachon o'zgartirgan
```

Yaxshi commit xabari

 Yomon

 Yaxshi

fix

Login formasida bo'sh parol bilan yuborish xatosi tuzatildi

update

Mahsulot kartasiga chegirma belgisi qo'shildi

asdasd

feat(cart): promokod qo'llash funksiyasi

Ko'plab jamoalar **Conventional Commits** uslubidan foydalanadi: `feat:` (yangi funksiya), `fix:` (xato tuzatish), `docs:`, `style:`, `refactor:`, `test:`, `chore:`. Qoida: bitta commit — bitta mantiqiy o'zgarish.

.gitignore

Ba'zi fayllar repozitoriyaga **hech qachon** tushmasligi kerak:

.gitignore

```
# Bog'liqliklar (npm install bilan qayta tiklanadi)
node_modules/

# Maxfiy ma'lumotlar — PAROLLAR, KALITLAR!
.env
.env.local
*.pem

# Build natijalari
dist/
build/

# Log va tizim fayllari
*.log
.DS_Store
Thumbs.db

# Muharrir sozlamalari
.vscode/
.idea/
```



Maxfiy kalitni push qilib qo'ydingizmi?

Faylni o'chirib, yangi commit qilish yetarli emas — kalit tarixda qoladi. GitHub'dagi botlar ochiq repozitoriyalardagi kalitlarni bir necha daqiqada topadi. **Darhol kalitni bekor qiling (revoke) va yangisini yarating.**

Branch'lar bilan ishlash

Terminal

```
git branch                # branch'lar ro'yxati (* – joriy)
git branch feature/cart   # yangi branch yaratish
git switch feature/cart   # unga o'tish (eski usul: git checkout
feature/cart)
git switch -c feature/search # yaratish + o'tish bir buyruqda

# ... o'zgarishlar, commit'lar ...

git switch main           # asosiy branch'ga qaytish
git merge feature/cart    # feature/cart ni main ga birlashtirish
git branch -d feature/cart # birlashtirilgan branch'ni o'chirish
```

Merge turlari

- **Fast-forward** — `main` da yangi commit bo'lmagan bo'lsa, Git shunchaki ko'rsatkichni oldinga suradi. Tarix to'g'ri chiziq.
- **Uch tomonlama merge** — ikkala branch'da ham yangi commit'lar bo'lsa, Git yangi **merge commit** yaratadi.
- **Rebase** — `git rebase main` — sizning commit'laringizni `main` ning oxiriga "ko'chirib o'tkazadi", tarix chiziqli bo'ladi. **Qoida:** boshqalar bilan bo'lishilgan (push qilingan) branch'ni rebase qilmang.

Konfliktlar (merge conflict)

Ikki branch bitta faylning **bitta qatorini** turlicha o'zgartirsa, Git qaysi birini tanlashni bilmaydi va sizdan so'raydi:

index.html

```
<<<<<<< HEAD
<h1>Xush kelibsiz, aziz mijoz!</h1>
=====
<h1>Bizning do'konga xush kelibsiz!</h1>
>>>>>> feature/new-header
```

Hal qilish tartibi:

1. Faylni oching; `<<<<<< HEAD` va `=====` orasida — sizning versiyangiz, `=====` va `>>>>>>` orasida — birlashtirilayotgan branch versiyasi.
2. Keraklisini qoldiring (yoki ikkalasini birlashtiring) va **barcha belgilarni o'chiring**.
3. `git add index.html` — "konflikt hal qilindi" deb belgilash.
4. `git commit` — merge'ni yakunlash.

VS Code konfliktlarni rangli ko'rsatadi va "Accept Current / Incoming / Both" tugmalarini taklif qiladi.
Bekor qilish: `git merge --abort`.

Xatolarni tuzatish va orqaga qaytish

Vaziyat	Buyruq
Fayldagi saqlanmagan o'zgarishlarni bekor qilish	<code>git restore app.js</code>
Faylni staging'dan chiqarish (o'zgarish qoladi)	<code>git restore --staged app.js</code>
Oxirgi commit xabarini o'zgartirish / unutilgan faylni qo'shish (push qilinmagan bo'lsa)	<code>git commit --amend</code>
Oxirgi commit'ni bekor qilish, o'zgarishlarni saqlab qolish	<code>git reset --soft HEAD~1</code>
Push qilingan commit'ni xavfsiz bekor qilish (teskari commit yaratadi)	<code>git revert a3f9c2e</code>
Ishni vaqtincha chetga qo'yish (boshqa branch'ga o'tish uchun)	<code>git stash</code> → keyin <code>git stash pop</code>
Eski versiyadagi faylni ko'rish	<code>git show a3f9c2e:src/app.js</code>
"Yo'qolgan" commit'ni topish	<code>git reflog</code>



`git reset --hard` va `git push --force` — ma'lumotni qaytarib bo'lmas darajada o'chirishi mumkin bo'lgan buyruqlar. Umumiy branch'larda `revert` dan foydalaning; force push kerak bo'lsa `--force-with-lease` xavfsizroq.

Remote: GitHub bilan ishlash

Terminal

```
# Mavjud repozitoriyani yuklab olish
git clone https://github.com/oybekalloyorov/web_fullstak.git
cd web_fullstak

# Lokal repozitoriyani GitHub'ga ulash
git remote add origin https://github.com/USERNAME/REPO.git
git remote -v                # ulangan remote'lar

# Yuborish
git push -u origin main      # -u: keyingi safar shunchaki "git push"
git push origin feature/cart # boshqa branch

# Olish
git fetch origin              # o'zgarishlarni yuklab olish
(birlashtirmasdan)
git pull                      # fetch + merge
git pull --rebase              # fetch + rebase (tarix toza qoladi)
```

Autentifikatsiya

GitHub parol bilan push qilishni qabul qilmaydi. Ikki usul bor:

- **HTTPS + Personal Access Token** (Settings → Developer settings → Tokens) yoki GitHub CLI: `gh auth login`.
- **SSH kalit:** `ssh-keygen -t ed25519 -C "siz@example.com"`, so'ng `~/.ssh/id_ed25519.pub` mazmunini GitHub → Settings → SSH keys ga qo'shing. Remote: `git@github.com:USERNAME/REPO.git`.

Jamoaviy ish oqimi: GitHub Flow

1. `main` — doim ishlaydigan, deploy qilinadigan kod.
2. Har bir vazifa uchun **yangi branch**: `feature/payment`, `fix/login-error`.
3. Kichik, mantiqiy commit'lar; muntazam `push`.
4. GitHub'da **Pull Request (PR)** ochish: nima va nima uchun o'zgartirilganini yozish.
5. **Code review**: jamoadoshlar kodni ko'rib chiqadi, izoh qoldiradi; avtomatik testlar (CI) ishlaydi.

6. Tasdiqlangach — `main` ga merge va branch'ni o'chirish.

7. Deploy.



Fork va open-source

Boshqa birovning loyihasiga hissa qo'shish uchun: repozitoriyaning **Fork** qiling (o'z akkauntingizga nusxa) → clone → branch → o'zgarish → push (o'z fork'ingizga) → asl loyihaga Pull Request oching.

Amaliy misollar

MISOL 1 Noldan loyiha yaratish va GitHub'ga joylash

Terminal

```
# 1. Loyiha papkasi
mkdir portfolio && cd portfolio
git init

# 2. Fayllar
echo "# Mening portfoliom" > README.md
echo "node_modules/" > .gitignore
touch index.html style.css

# 3. Birinchi commit
git status          # 4 ta "untracked" fayl
git add .
git commit -m "Loyiha tuzilmasi yaratildi"

# 4. GitHub'da bo'sh repozitoriya yarating (README'siz!), so'ng:
git branch -M main
git remote add origin https://github.com/USERNAME/portfolio.git
git push -u origin main

# 5. Keyingi ish sikli
#   (index.html ga navigatsiya qo'shdingiz)
git add index.html
git commit -m "feat: sarlavha va navigatsiya qo'shildi"
#   (style.css ga dizayn)
git add style.css
git commit -m "style: navbar uchun flex maket"
git push

git log --oneline
# 9c1e4b2 style: navbar uchun flex maket
# 5a7d310 feat: sarlavha va navigatsiya qo'shildi
# 1f0a2c8 Loyiha tuzilmasi yaratildi
```

Siz ushbu qo'llanmaning repozitoriyasini ham aynan shunday boshlagansiz: `git init` → `git add README.md` → `git commit` → `git remote add origin` → `git push -u origin main`.

MISOL 2 Feature branch, Pull Request va code review

Terminal

```
# main yangi ekaniga ishonch hosil qilamiz
git switch main
git pull

# Vazifa: "Aloqa formasi" – yangi branch
git switch -c feature/contact-form

# ... contact.html yaratildi ...
git add contact.html
git commit -m "feat(contact): aloqa formasi sahifasi"

# ... validatsiya qo'shildi ...
git add js/contact.js
git commit -m "feat(contact): email va telefon validatsiyasi"

git push -u origin feature/contact-form
# Terminal PR yaratish havolasini ko'rsatadi:
# remote: Create a pull request for 'feature/contact-form' on GitHub by
# visiting:
# remote:
https://github.com/USERNAME/portfolio/pull/new/feature/contact-form
```

GitHub'da PR tavsifiga namuna:

Matn

```
## Nima o'zgardi
- /contact sahifasi va aloqa formasi qo'shildi
- Email va +998 formatidagi telefon raqami tekshiriladi

## Qanday tekshirish mumkin
1. /contact.html ni oching
2. Noto'g'ri email kiriting – qizil xabar chiqishi kerak

## Skrinshotlar
(rasm)

Closes #12
```

Reviewer izoh qoldirdi: "Telefon maydoniga `autocomplete="tel"` qo'shing". Siz shu branch'da tuzatasiz:

Terminal

```
git add contact.html
git commit -m "fix(contact): telefon maydoniga autocomplete qo'shildi"
git push                                # PR avtomatik yangilanadi

# PR merge qilingach:
git switch main
git pull
git branch -d feature/contact-form
git push origin --delete feature/contact-form # (GitHub'da "Delete
branch" tugmasi ham bor)
```

Closes #12 — PR merge qilinganda 12-raqamli issue avtomatik yopiladi.

MISOL 3 Konfliktni qadamma-qadam hal qilish

Konfliktni o'z kompyuteringizda ataylab yaratib, hal qilishni mashq qilamiz:

Terminal

```
mkdir conflict-demo && cd conflict-demo && git init
echo "Narx: 100 000 so'm" > price.txt
git add . && git commit -m "Boshlang'ich narx"

# Birinchi dasturchi: chegirma
git switch -c discount
echo "Narx: 80 000 so'm (chegirma!)" > price.txt
git commit -am "Chegirmali narx"

# Ikkinchi dasturchi (main'da): narx oshdi
git switch main
echo "Narx: 120 000 so'm" > price.txt
git commit -am "Narx oshirildi"

# Birlashtirish
git merge discount
# CONFLICT (content): Merge conflict in price.txt
# Automatic merge failed; fix conflicts and then commit the result.

git status          # both modified: price.txt
cat price.txt
```

Fayl mazmuni:

Matn

```
<<<<<< HEAD
Narx: 120 000 so'm
=====
Narx: 80 000 so'm (chegirma!)
>>>>>> discount
```

Jamoa bilan kelishib, yangi narxga chegirma qo'llaymiz. Faylni shunday tahrirlaymiz:

Matn

```
Narx: 96 000 so'm (120 000 dan 20% chegirma!)
```

Terminal

```
git add price.txt
git commit -m "Merge discount: yangi narxga chegirma qo'llandi"
git log --oneline --graph
# *    d4e5f6a Merge discount: yangi narxga chegirma qo'llandi
# | \
# | * b2c3d4e Chegirmali narx
# * | c3d4e5f Narx oshirildi
# | /
# * a1b2c3d Boshlang'ich narx
```

MISOL 4

Saytni GitHub Pages'da bepul joylash

Statik sayt (HTML/CSS/JS) uchun eng oson hosting — GitHub Pages. Ushbu qo'llanma ham shu usulda joylanadi.

1. Repozitoriyaning ildizida `index.html` bo'lsin va `main` ga push qiling.
2. GitHub'da repozitoriya → **Settings** → **Pages**.
3. **Source**: "Deploy from a branch", **Branch**: `main`, papka: `/ (root)` → Save.
4. 1–2 daqiqadan so'ng sayt `https://USERNAME.github.io/REPO/` manzilida ochiladi.

Avtomatlashtirilgan variant — GitHub Actions bilan har push'da deploy:

.github/workflows/pages.yml

```
name: Deploy to GitHub Pages
on:
  push:
    branches: [main]
permissions:
  contents: read
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: .
      - id: deployment
        uses: actions/deploy-pages@v4
```

Bunda Settings → Pages → Source sifatida "GitHub Actions" tanlanadi. O'z domeningiz bo'lsa (masalan `portfolio.uz`), Pages sozlamasida "Custom domain" ga yozing va domen DNS'ida `CNAME` yozuvini `USERNAME.github.io` ga yo'naltiring (1-darsdagi DNS bo'limi).

Mashqlar



Mustaqil ish

1. Oldingi darslarda yozgan portfolio va Todo ilovangizni alohida repoziitoriyalarga joylang va GitHub Pages orqali e'lon qiling.
2. Learn Git Branching interaktiv darslarining "Main" bo'limini o'ting.

3. 3-misoldagi konfliktni takrorlang, lekin bu safar `git merge` o'rniga `git rebase main` bilan. Tarixda qanday farq bor?
4. Do'stingiz bilan bitta repozitoriyada ishlang: har biringiz alohida branch'da bitta faylni o'zgartiring, PR oching, bir-biringizning kodini review qiling.
5. Ataylab xato commit qiling va uni (a) `--amend`, (b) `reset --soft`, (c) `revert` bilan tuzating. Qaysi biri push'dan keyin xavfsiz?

✓ Xulosa

- Git — lokal versiyalar tizimi; GitHub — uning ustidagi hamkorlik platformasi.
- Ish sikli: o'zgartirish → `git add` → `git commit` → `git push`.
- Har bir vazifa — alohida branch; `main` doim ishlaydigan holatda.
- Konflikt — qo'rqinchli emas: belgilarni o'chirib, to'g'ri variantni qoldirib, `add` + `commit`.
- `.env` va `node_modules` — doim `.gitignore` da.
- Pull Request + code review — professional jamoaviy ishning standarti.

□ Darsni o'qib bo'ldim